

Reflective Report

CA One — Programming for Data Analysis (B9DA108)

Osamuyimen Osunde

MSc in Data Analytics, Dublin Business School

Introduction

This report walks through how I approached the three parts of CA One, the savings planner, the inventory system, and the timetabling model, and what I took from building each one. I have tried to write it the way I actually worked through the problems rather than presenting it as though everything came together neatly on the first attempt, because it didn't.

My background is in front-end development, so a lot of my instinct going into this assignment was to think in components, props, and state rather than financial models or scheduling rules. A good part of the effort here, then, was less about Python syntax and more about learning to reason about a new kind of problem, one where the correctness of the logic matters more than how it looks.

Task 1 — Financial Planning and Savings Optimisation System

The brief for this task was to model how an individual's savings grow month by month, given a starting salary, a fixed portion of income saved, a 4% annual return on savings, and annual salary growth of somewhere between 2% and 8%. The order of operations mattered: interest is applied to whatever is already in the account first, and only then is that month's contribution added on top.

Simulating the savings behaviour

I wrote `simulate_savings()` as a straightforward monthly loop that applies interest, adds the monthly contribution, and, every twelfth iteration, grows the annual salary. Running it with a starting salary of €40,000, a 15% savings rate, and 3% annual growth gave a balance of €12,654.78 after 24 months. That number on its own doesn't mean much, but it became the baseline I checked every later function against, since any change to the underlying logic should still reproduce it.

Input validation was something I initially skipped and came back to later, once I realised how easily a negative salary or a savings rate above 1 could quietly produce nonsense results instead of an obvious error. Raising a `ValueError` with a specific message felt like the more honest approach, better to fail loudly at the point where the bad input enters than to debug a strange chart three cells later.

Time to reach a target

`time_to_target()` reuses the same monthly mechanics but checks the balance after every iteration and stops the moment it crosses the target, rather than running for a fixed number of months and reading the answer off afterwards. With the same starting figures and a €20,000 target, the function returns 37 months. I also capped the loop at 600 months, on the reasoning that a target which isn't reachable inside about fifty years is, for practical purposes, not reachable at all, and returning `None` felt clearer than letting the loop run indefinitely.

Finding the optimal savings rate

This was the part of Task 1 I found most interesting. Fixing the horizon at 24 months and asking for the savings rate needed to hit a target within €100 is really a root-finding problem, and I used bisection search rather than trying every rate in turn. The reasoning is that final balance increases steadily as the savings rate increases —

there's no point where saving more results in less money, so once you know a rate of 0% falls short and a rate of 100% overshoots, the correct rate has to sit somewhere between the two, and each check of the midpoint halves the space you still need to search.

For a target of €15,000 over 24 months, this converges on a savings rate of roughly 17.8%, producing a final balance of €14,994.60, comfortably inside the €100 tolerance. It took noticeably fewer iterations than I expected, which is really the point of bisection over a brute-force sweep: instead of testing every possible rate, you only need on the order of $\log_2(n)$ checks to close in on the answer.

Visualising the results

I plotted the 24-month growth curve at the optimal rate, and separately compared several fixed savings rates against each other on the same axes. The second chart made the impact of the savings rate more obvious than any single number could, the gap between a 5% saver and a 20% saver over the same two years is much larger than I'd have guessed before actually plotting it.

Reflection

The main lesson from this task was less about the Python and more about recognising when a problem has a structure that a specific algorithm is suited to. My first instinct was to loop through candidate rates in small steps and stop when I got close enough, which works but is wasteful once the range is fine enough to matter. Realising the balance-versus-rate relationship was monotonic was what made bisection the obvious choice rather than a textbook one.

Task 2 — Retail Inventory Management System

Choosing a data structure

The brief asked for a structure that prevents duplicate entries and scales to hundreds of products, so I went with a dictionary keyed on `product_id` rather than a list of product records. With a list, checking for a duplicate before every insert means scanning the whole collection, and the same is true for updates or deletions by ID, the cost grows with the number of products. A dictionary gives near-constant-time lookup, insertion, and deletion by key, which matters more here than it would for a handful of items, since the brief is explicit that the system needs to handle a much larger catalogue without slowing down.

I used a `Product` dataclass to hold the five required fields, mainly for the free `__init__` and readable `__repr__` it gives, it doesn't change the underlying complexity argument, but it made the rest of the code easier to read while I was writing it.

Managing the inventory

`InventoryManager` wraps the dictionary and exposes `add`, `update`, `delete`, and `retrieve` operations, each checking its inputs before touching the data: empty identifiers are rejected, negative prices or quantities are rejected, and attempting to add a `product_id` that already exists raises a clear error instead of silently overwriting the original entry. Testing this by deliberately trying to add a duplicate `P001` confirmed it behaves as intended, the second attempt is blocked with a message naming the conflicting ID rather than a generic failure.

Analysis with pandas

Per the brief, the inventory is written out to a CSV file first, and every piece of analysis that follows reads from that file rather than from the dictionary still sitting in memory, the two are kept properly separate. Grouping by category gave total stock, average unit price, and inventory value per category; filtering for anything below a

threshold of 10 units flagged three products as needing reorder out of eight in total; and the overall summary put total inventory value at €7,229.26, with Electronics coming out as the most valuable category.

Visualisation

Two charts came out of the CSV-driven analysis: a side-by-side comparison of stock levels and inventory value by category, and a horizontal bar chart of the low-stock items with the reorder threshold marked as a reference line, which made it easy to see at a glance how far below the line each flagged product actually sits.

Reflection

This task reinforced something that isn't always obvious from a textbook description of Big-O notation: the difference between $O(1)$ and $O(n)$ lookups only really becomes a meaningful design decision once you think about how many times an operation will run and at what scale. For eight products, a list would have worked fine and nobody would notice; the brief's emphasis on scaling to hundreds of items is what made the dictionary the right call rather than just a defensible one.

Task 3 — DBS Timetabling and Enrolment System

This task asked for four related classes: Venue, Module, Lecturer, and Student, with each one responsible for its own slice of conflict checking. Working out where each check should live turned out to be the real design problem, more so than any individual method.

Venue and scheduling conflicts

`Venue.addModule()` checks whether its own timetable already has something booked in that day/time slot before accepting a new module, raising a `SchedulingConflictError` if it does. Keeping this check inside the venue itself, rather than in some central scheduler, meant a room only ever needs to know about its own bookings, not the whole college's.

Module, Lecturer, and availability

`Module.setLecturer()` only succeeds if the lecturer actually teaches that subject and is free at the module's current time; `Module.setTimetable()` checks the assigned lecturer's availability before moving the module to a new slot. Lecturer objects track their own commitments and expose an `is_available()` check, so a Module never needs to reach into another lecturer's diary to figure out whether an assignment is safe.

Student enrolment

`Student.addModule()` checks two things before enrolling: whether the module still has capacity, and whether the new module clashes with anything already on the student's own timetable. Both checks live on the student side rather than the module side, since it's the student's schedule that a clash actually affects.

Worked example

I built a small scenario to exercise all four classes together: a Programming for Data Analysis module assigned to Dr Smith on Monday at 10:00 in room A101, with a capacity of two. Trying to book a second class into the same room at the same time correctly raised a conflict error, and trying to assign Dr Smith to a Statistics class in that same slot correctly failed because he was already committed elsewhere. Two students enrolled successfully; a third was rejected once the module hit capacity, and a fourth attempt, trying to add a clashing Statistics module to a student already in Programming for Data Analysis at the same time, was also rejected. Seeing each of these

fail for the right reason, rather than just not crashing, was the most useful check that the logic was actually doing what it was supposed to.

Reflection

What stood out here was how much cleaner the design became once I stopped trying to write one big function that checked everything at once and instead let each object be responsible only for what it directly owns. It's a fairly small system, but the same pattern, an object validating against its own state rather than an external coordinator validating everything, is one I recognise from component-based front-end work, even though the domain here is completely different.

Critical Evaluation and Limitations

A few limitations are worth naming honestly rather than glossing over. The bisection search in Task 1 depends on the relationship between savings rate and final balance being monotonic, which holds under the assumptions given here but wouldn't necessarily hold if, for instance, the interest rate itself changed with the account balance. The inventory system's category-wise analysis is fine at the scale used in this assignment, but a genuinely large catalogue would benefit from indexing rather than a full pandas groupby on every query. And the timetabling model only checks for exact slot matches, two classes that overlap partially, say one running 10:00–11:30 and another starting at 11:00, wouldn't currently be flagged as a conflict, since the comparison is on the discrete (day, time) pair rather than a true time range.

Given more time I'd have liked to extend the timetabling system to handle interval overlap properly, and to add persistence for Task 3 the way Task 2 already has with CSV export, so a timetable could be saved and reloaded rather than existing only for the length of a single session.

Conclusion

Across the three tasks, the common thread for me was less about any single algorithm and more about matching the shape of a problem to a sensible structure, a dictionary where lookups matter, bisection where a relationship is monotonic, and small self-contained objects where conflict checking needs to stay local rather than centralised. Coming from a front-end background, that last point in particular felt familiar even in an unfamiliar domain, and it's the part of this assignment I expect to carry forward most directly into the rest of the MSc.